RAJALAKSHMI ENGINEERING COLLEGE

Thandalam, Chennai – 602015

DEPARTMENT OF COMPUTER SCIENCE AND DESIGN

OPERATING SYSTEMS LABORATORY RECORD

CS23431 – OPERATING SYSTEMS

Regulation 2023

Name	RAM.A.M
Register No.	2116241701502
Academic Year	2026 - 2027
Semester	5TH

TABLE OF CONTENTS

EXP. NO.	EXPERIMENT	DATE	SIGNATURE
1	Shell Script – Arithmetic Operation using expr command		
2	Shell Script – Reverse Number, Leap Year and Fibonacci Series		
3	Text Processing using AWK Script		
4	System Calls – fork(), exec(), getpid(), opendir(), readdir()		
5	CPU Scheduling – FCFS, SJF, Priority and Round Robin		
6	Inter-Process Communication using Shared Memory		
7	Producer Consumer using Semaphores		
8	Banker’s Deadlock Avoidance Algorithm		
9	Contiguous Memory Allocation – Best Fit		
10	Contiguous Memory Allocation – First Fit		
11	Page Replacement – FIFO		
12	Page Replacement – LRU		
13	Page Replacement – Optimal		

EXPERIMENT 1

SHELL SCRIPT – ARITHMETIC OPERATION USING EXPR COMMAND

Date: _____

AIM

To write a shell script to display basic arithmetic operations using the expr command.

ALGORITHM / PROCEDURE

1. Read two numbers from the user.

2. Use expr to calculate addition, subtraction, multiplication, division and modulus.

3. Display the calculated values.

PROGRAM

```
echo "enter two no"
read a
read b
c=`expr $a + $b`
d=`expr $a - $b`
e=`expr $a \* $b`
f=`expr $a / $b`
g=`expr $a % $b`
echo "add $c"
echo "sub $d"
echo "mul $e"
echo "div $f"
echo "mod $g"
```

OUTPUT

```
$ sh arith.sh
Enter two no
5
10
add 15
sub -5
mul 50
div 0
mod 5
```

RESULT

Thus, the shell script was executed and the required arithmetic operations were performed using the expr command.

EXPERIMENT 2

SHELL SCRIPT – REVERSE NUMBER, LEAP YEAR AND FIBONACCI SERIES

Date: _____

AIM

To write shell scripts for reversing a number, checking a leap year using if-else, and generating a Fibonacci series using a for loop.

ALGORITHM / PROCEDURE

1. Read the input number.

2. For reverse operation, repeatedly obtain the last digit using modulus and build the reversed number.

3. For leap year checking, calculate the remainder when the year is divided by 4 and use a conditional statement.

4. For Fibonacci series, initialize the first two values and generate subsequent values using a loop.

PROGRAM

```
# Reverse a number
echo "enter number"
read n
rev=0
while [ $n -gt 0 ]
do
d=`expr $n % 10`
rev=$(echo "$rev * 10 + $d"|bc)
n=`expr $n / 10`
done
echo $rev

# Leap year
echo "enter number"
read a
m=`expr $a % 4`
if [ $m -eq 0 ]
then
echo "leap year"
else
echo "not leap year"
fi

# Fibonacci series
echo "enter number"
read n
a=-1
b=1
echo "fibonacci series"
for((i=1;i<=$n;i++))
do
c=$(echo "$a + $b"|bc)
echo $c
a=$b
b=$c
done
```

OUTPUT

```
Reverse:
enter number
123
321

Leap year:
enter number
12
leap year

Fibonacci:
```

```
0
1
1
2
3
5
8
13
21
34
```

RESULT

Thus, the required shell-script operations for reversing a number, checking a leap year and generating Fibonacci numbers were implemented.

EXPERIMENT 3

TEXT PROCESSING USING AWK SCRIPT

Date: _____

AIM

To process text files using AWK and obtain employee average pay and examination pass/fail status.

ALGORITHM / PROCEDURE

1. Create the input data file.

2. Read each record using an AWK script.

3. For employee data, select employees whose salary is greater than 6000 and whose days worked are more than 4, then calculate total and average pay.

4. For examination results, compare all subject marks with the pass mark and display PASS or FAIL.

PROGRAM

```
# emp.dat
JOE 8000 5
RAM 6000 5
TIM 5000 6
BEN 7000 7
AMY 6500 6

# emp.awk
BEGIN { print "EMPLOYEES DETAILS" }
{
    if($2>6000 && $3>4)
    {
        print $1,"\t\t",$2*$3
        pay=pay+$2*$3
        count=count+1
    }
}
END {
    print "no of employees are=",count
    print "total pay=",pay
    print "average pay=",pay/count
}

# marks.dat
BEN 40 55 66 77 55 77
TOM 60 67 84 92 90 60
RAM 90 95 84 87 56 70
JIM 60 70 65 78 90 87

# marks.awk
BEGIN {
    print "NAME SUB-1 SUB-2 SUB-3 SUB-4 SUB-5 SUB-6 STATUS"
}
{
    if ($2 < 45 || $3 < 45 || $4 < 45 || $5 < 45 || $6 < 45 || $7 < 45)
        print $1,"\t",$2,"\t",$3,"\t",$4,"\t",$5,"\t",$6,"\t",$7,"\tFAIL"
    else
        print $1,"\t",$2,"\t",$3,"\t",$4,"\t",$5,"\t",$6,"\t",$7,"\tPASS"
}
```

OUTPUT

```
EMPLOYEES DETAILS
JOE 40000
BEN 49000
AMY 39000
total pay = 128000
average pay = 42666.7

BEN 40 55 66 77 55 77 FAIL
TOM 60 67 84 92 90 60 PASS
RAM 90 95 84 87 56 70 PASS
```

```
JIM 60 70 65 78 90 87 PASS
```

RESULT

Thus, AWK scripts were used successfully for employee pay analysis and examination result processing.

EXPERIMENT 4

SYSTEM CALLS – FORK(), EXEC(), GETPID(), OPENDIR(), READDIR()

Date: _____

AIM

To experiment with basic Linux system calls including process creation, process identification, program execution and directory handling.

ALGORITHM / PROCEDURE

1. Include the required system headers.

2. Create a child process using fork().

3. Use getpid() and getppid() to display process identifiers.

4. Use an exec-family function to replace a process image with another program.

5. Use opendir() to open a directory and readdir() to read directory entries.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <dirent.h>

int main()
{
    int pid;
    DIR *dir;
    struct dirent *entry;

    pid = fork();
    printf("\nTHIS LINE EXECUTED TWICE");

    if(pid == -1)
    {
        printf("\nCHILD PROCESS NOT CREATED\n");
        exit(0);
    }

    if(pid == 0)
    {
        printf("\nI AM CHILD PROCESS AND MY ID IS %d\n", getpid());
        printf("THE CHILD PARENT PROCESS ID IS %d\n", getppid());
    }
    else
    {
        printf("\nI AM PARENT PROCESS AND MY ID IS %d\n", getpid());
        printf("THE PARENT'S PARENT PROCESS ID IS %d\n", getppid());
    }

    dir = opendir(".");
    if(dir != NULL)
    {
        printf("\nDirectory entries:\n");
        while((entry = readdir(dir)) != NULL)
            printf("%s\n", entry->d_name);
        closedir(dir);
    }

    return 0;
}
```

OUTPUT

```
THIS LINE EXECUTED TWICE
I AM CHILD PROCESS AND MY ID IS 2145
THE CHILD PARENT PROCESS ID IS 2144
```

```
Directory entries:
.
..
program.c

I AM PARENT PROCESS AND MY ID IS 2144
THE PARENT'S PARENT PROCESS ID IS 1800
```

RESULT

Thus, process creation and identification were demonstrated, and directory entries were accessed using opendir() and readdir().

EXPERIMENT 5

CPU SCHEDULING – FCFS, SJF, PRIORITY AND ROUND ROBIN

Date: _____

AIM

To implement and study the CPU scheduling techniques FCFS, SJF, Priority and Round Robin.

ALGORITHM / PROCEDURE

1. Read the number of processes and their scheduling information.

2. For FCFS, execute processes in arrival order and calculate waiting and turnaround times.

3. For SJF, select the process with the shortest burst time.

4. For Priority scheduling, select the process according to its priority.

5. For Round Robin, allocate a fixed time quantum to each ready process repeatedly.

6. Display waiting time, turnaround time and average values.

PROGRAM

```
FCFS – Python implementation
bt=[]
n=int(input("Enter the number of process: "))
bt=list(map(int, input("Enter burst times: ").split()))
wt=[0]
tat=[bt[0]]
for i in range(1,len(bt)):
    wt.append(wt[i-1]+bt[i-1])
    tat.append(wt[i]+bt[i])
print("Process\tBurst\tWaiting\tTurnaround")
for i in range(n):
    print(i, bt[i], wt[i], tat[i])
print("Average Waiting Time:", sum(wt)/n)
print("Average Turn Around Time:", sum(tat)/n)

Scheduling methods:
FCFS       -> First process to arrive is executed first.
SJF        -> Shortest burst-time process is selected.
Priority   -> Highest-priority process is selected.
Round Robin -> Each process receives a fixed time quantum.
```

OUTPUT

```
FCFS example
Enter the number of process:
3
Enter the burst times:
24 3 3

Process  Burst  Waiting  Turnaround
0        24     0        24
1        3      24       27
2        3      27       30

Average Waiting Time: 17.0
Average Turn Around Time: 19.0
```

RESULT

Thus, the CPU scheduling techniques FCFS, SJF, Priority and Round Robin were studied and their scheduling principles were implemented.

EXPERIMENT 6

INTER-PROCESS COMMUNICATION USING SHARED MEMORY

Date: _____

AIM

To write a C program to perform inter-process communication using shared memory between sender and receiver processes.

ALGORITHM / PROCEDURE

1. Set the size and key of the shared memory segment.

2. Allocate the shared memory using shmget().

3. Attach the segment using shmat().

4. The sender writes a message into the shared memory.

5. The receiver reads the message from the shared memory.

6. Detach the shared memory using shmdt().

PROGRAM

```
/* sender.c */
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define SharedMemSize 50

int main()
{
    int shmid;
    key_t key = 5677;
    char *shared_memory;

    shmid = shmget(key, SharedMemSize, IPC_CREAT | 0666);
    if(shmid < 0) { perror("shmget"); exit(1); }

    shared_memory = shmat(shmid, NULL, 0);
    if(shared_memory == (char *)-1) { perror("shmat"); exit(1); }

    sprintf(shared_memory, "Welcome to Shared Memory");
    sleep(2);
    return 0;
}

/* receiver.c */
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#include <stdlib.h>

#define SharedMemSize 50

int main()
{
    int shmid;
    key_t key = 5677;
    char *shared_memory;

    shmid = shmget(key, SharedMemSize, 0666);
    if(shmid < 0) { perror("shmget"); exit(1); }

    shared_memory = shmat(shmid, NULL, 0);
    if(shared_memory == (char *)-1) { perror("shmat"); exit(1); }
    printf("Message Received: %s\n", shared_memory);
```

```
    return 0;
}
```

OUTPUT

```
Terminal 1
$ gcc sender.c -o sender
$ ./sender

Terminal 2
$ gcc receiver.c -o receiver
$ ./receiver
Message Received: Welcome to Shared Memory
```

RESULT

Thus, inter-process communication using shared memory was successfully implemented between sender and receiver processes.

EXPERIMENT 7

PRODUCER CONSUMER USING SEMAPHORES

Date: _____

AIM

To implement the solution to the producer-consumer problem using semaphore variables.

ALGORITHM / PROCEDURE

1. Initialize mutex, full and empty values.

2. Select producer, consumer or exit operation.

3. The producer decreases empty, increases full and produces an item.

4. The consumer decreases full, increases empty and consumes an item.

5. Prevent production when the buffer is full and prevent consumption when the buffer is empty.

PROGRAM

```
#include<stdio.h>
#include<stdlib.h>

int mutex=1, full=0, empty=3, x=0;

int wait(int s) { return --s; }
int signal(int s) { return ++s; }

void producer()
{
    mutex=wait(mutex);
    full=signal(full);
    empty=wait(empty);
    x++;
    printf("\nProducer produces the item %d",x);
    mutex=signal(mutex);
}

void consumer()
{
    mutex=wait(mutex);
    full=wait(full);
    empty=signal(empty);
    printf("\nConsumer consumes item %d",x);
    x--;
    mutex=signal(mutex);
}

int main()
{
    int n;
    printf("\n1.Producer\n2.Consumer\n3.Exit");

    while(1)
    {
        printf("\nEnter your choice:");
        scanf("%d",&n);

        switch(n)
        {
            case 1:
                if((mutex==1)&&(empty!=0)) producer();
                else printf("Buffer is full!!");
                break;
            case 2:
                if((mutex==1)&&(full!=0)) consumer();
                else printf("Buffer is empty!!");
                break;
            case 3:
                exit(0);
        }
```

```
    }
}
```

OUTPUT

```
1.Producer
2.Consumer
3.Exit
Enter your choice:1
Producer produces the item 1
Enter your choice:2
Consumer consumes item 1
Enter your choice:2
Buffer is empty!!
Enter your choice:1
Producer produces the item 1
Enter your choice:1
Producer produces the item 2
Enter your choice:1
Producer produces the item 3
Enter your choice:1
Buffer is full!!
```

RESULT

Thus, the producer-consumer problem was implemented and synchronization between producer and consumer was demonstrated.

EXPERIMENT 8

BANKER'S DEADLOCK AVOIDANCE ALGORITHM

Date: _____

AIM

To find out a safe sequence using Banker’s algorithm for deadlock avoidance.

ALGORITHM / PROCEDURE

1. Initialize Work with Available and Finish for all processes as false.

2. Find a process whose Need is less than or equal to Work.

3. If found, add its Allocation to Work and mark the process finished.

4. Repeat until all processes are finished or no suitable process remains.

5. If all processes finish, display the safe sequence; otherwise report that no safe sequence exists.

PROGRAM

```
#include <stdio.h>

int main()
{
    int n=5, m=3, i, j, k;
    int alloc[5][3]={{0,1,0},{2,0,0},{3,0,2},{2,1,1},{0,0,2}};
    int max[5][3]={{7,5,3},{3,2,2},{9,0,2},{2,2,2},{4,3,3}};
    int avail[3]={3,3,2};
    int f[5]={0}, ans[5], ind=0, need[5][3];

    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            need[i][j]=max[i][j]-alloc[i][j];

    for(k=0;k<n;k++)
        for(i=0;i<n;i++)
            if(f[i]==0)
            {
                int flag=0;
                for(j=0;j<m;j++)
                    if(need[i][j]>avail[j]) { flag=1; break; }

                if(flag==0)
                {
                    ans[ind++]=i;
                    for(j=0;j<m;j++)
                        avail[j]+=alloc[i][j];
                    f[i]=1;
                }
            }

    printf("The SAFE Sequence is\n");
    for(i=0;i<n;i++)
        printf("P%d%s",ans[i],i==n-1?"":" -> ");
    return 0;
}
```

OUTPUT

```
The SAFE Sequence is
P1 -> P3 -> P4 -> P0 -> P2
```

RESULT

Thus, the Banker’s algorithm was implemented successfully and a safe sequence was obtained.

EXPERIMENT 9

CONTIGUOUS MEMORY ALLOCATION – BEST FIT

Date: _____

AIM

To implement the Best Fit memory allocation technique.

ALGORITHM / PROCEDURE

1. Read the sizes of available memory blocks and processes.

2. For each process, search all blocks that can accommodate it.

3. Select the smallest suitable block.

4. Allocate the process and reduce the remaining size of that block.

5. Display the process-to-block allocation.

PROGRAM

```
#include<stdio.h>
#include<string.h>

void bestFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];
    memset(allocation,-1,sizeof(allocation));

    for(int i=0;i<n;i++)
    {
        int bestIdx=-1;
        for(int j=0;j<m;j++)
            if(blockSize[j]>=processSize[i])
                if(bestIdx==-1 || blockSize[bestIdx]>blockSize[j])
                    bestIdx=j;

        if(bestIdx!=-1)
        {
            allocation[i]=bestIdx;
            blockSize[bestIdx]-=processSize[i];
        }
    }

    printf("\nProcess No.\tProcess Size\tBlock no.\n");
    for(int i=0;i<n;i++)
    {
        printf("%d\t\t%d\t\t",i+1,processSize[i]);
        if(allocation[i]!=-1) printf("%d",allocation[i]+1);
        else printf("Not Allocated");
        printf("\n");
    }
}

int main()
{
    int blockSize[]={100,500,200,300,600};
    int processSize[]={212,417,112,426};
    int m=sizeof(blockSize)/sizeof(blockSize[0]);
    int n=sizeof(processSize)/sizeof(processSize[0]);
    bestFit(blockSize,m,processSize,n);
    return 0;
}
```

OUTPUT

```
Process No.    Process Size    Block no.
1              212             4
2              417             2
3              112             3
4              426             5
```

RESULT

Thus, the Best Fit contiguous memory allocation technique was implemented successfully.

EXPERIMENT 10

CONTIGUOUS MEMORY ALLOCATION – FIRST FIT

Date: _____

AIM

To implement memory allocation using the First Fit technique.

ALGORITHM / PROCEDURE

1. Read the number and sizes of memory blocks and files.

2. For each file, scan blocks from the beginning.

3. Select the first unallocated block that is large enough.

4. Record the block number and fragmentation.

5. Display the allocation table.

PROGRAM

```
#include<stdio.h>
#define max 25

int main()
{
    int frag[max],b[max],f[max],i,j,nb,nf,temp;
    static int bf[max],ff[max];

    printf("\nEnter the number of blocks:");
    scanf("%d",&nb);
    printf("Enter the number of files:");
    scanf("%d",&nf);

    printf("\nEnter the size of the blocks:-\n");
    for(i=1;i<=nb;i++)
    {
        printf("Block %d:",i);
        scanf("%d",&b[i]);
    }

    printf("Enter the size of the files:-\n");
    for(i=1;i<=nf;i++)
    {
        printf("File %d:",i);
        scanf("%d",&f[i]);
    }

    for(i=1;i<=nf;i++)
    {
        for(j=1;j<=nb;j++)
        {
            if(bf[j]!=1)
            {
                temp=b[j]-f[i];
                if(temp>=0)
                {
                    ff[i]=j;
                    break;
                }
            }
        }
        frag[i]=temp;
        bf[ff[i]]=1;
    }

    printf("\nFile_no:\tFile_size:\tBlock_no:\tBlock_size:\tFragment");
    for(i=1;i<=nf;i++)
        printf("\n%d\t\t%d\t\t%d\t\t%d\t\t%d",
               i,f[i],ff[i],b[ff[i]],frag[i]);
    return 0;
}
```

OUTPUT

```
File_no:    File_size:    Block_no:    Block_size:    Fragment
1            ...           ...          ...            ...
2            ...           ...          ...            ...
3            ...           ...          ...            ...
```

RESULT

Thus, the First Fit memory allocation technique was implemented and the file allocation table was displayed.

EXPERIMENT 11

PAGE REPLACEMENT – FIFO

Date: _____

AIM

To find the number of page faults using the First-In First-Out (FIFO) page replacement technique.

ALGORITHM / PROCEDURE

1. Initialize the page frames.

2. Read the reference string.

3. Check whether each referenced page is already in memory.

4. If the page is absent, replace the oldest page using FIFO order.

5. Count and display the total page faults.

PROGRAM

```
def fifo():
    f=-1
    page_faults=0
    page=[-1]*m

    for i in range(n):
        flag=0
        for j in range(m):
            if page[j]==a[i]:
                flag=1
                break

        if flag==0:
            f=(f+1)%m
            page[f]=a[i]
            page_faults+=1
            print(a[i],"->",page)
        else:
            print(a[i],"-> No Page Fault")

    print("Total page faults:",page_faults)

a=[]
n=int(input("Enter the size of reference string: "))
for i in range(n):
    a.append(int(input("Enter page: ")))

m=int(input("Enter page frame size: "))
fifo()
```

OUTPUT

```
Enter the size of reference string : 20
Enter page frame size : 3
...
Total page faults: 15
```

RESULT

Thus, the FIFO page replacement algorithm was implemented successfully and the number of page faults was obtained.

EXPERIMENT 12

PAGE REPLACEMENT – LRU

Date: _____

AIM

To implement the Least Recently Used (LRU) page replacement algorithm.

ALGORITHM / PROCEDURE

1. Read the number of frames and the reference string.

2. Initialize all frames as empty.

3. If the referenced page is present, update its recent-use counter.

4. If it is absent and a free frame exists, insert the page.

5. Otherwise replace the page with the smallest recent-use counter.

6. Count and display page faults.

PROGRAM

```
#include<stdio.h>

int findLRU(int time[], int n)
{
    int i, minimum=time[0], pos=0;
    for(i=1;i<n;++i)
        if(time[i]<minimum)
        {
            minimum=time[i];
            pos=i;
        }
    return pos;
}

int main()
{
    int no_of_frames,no_of_pages,frames[10],pages[30];
    int counter=0,time[10],flag1,flag2,i,j,pos,faults=0;

    printf("Enter number of frames: ");
    scanf("%d",&no_of_frames);
    printf("Enter number of pages: ");
    scanf("%d",&no_of_pages);
    printf("Enter reference string: ");

    for(i=0;i<no_of_pages;++i) scanf("%d",&pages[i]);
    for(i=0;i<no_of_frames;++i) frames[i]=-1;

    for(i=0;i<no_of_pages;++i)
    {
        flag1=flag2=0;

        for(j=0;j<no_of_frames;++j)
            if(frames[j]==pages[i])
            {
                counter++;
                time[j]=counter;
                flag1=flag2=1;
                break;
            }

        if(flag1==0)
            for(j=0;j<no_of_frames;++j)
                if(frames[j]==-1)
                {
                    counter++;
                    faults++;
                    frames[j]=pages[i];
                    time[j]=counter;
```

```
                    flag2=1;
                    break;
                }

        if(flag2==0)
        {
            pos=findLRU(time,no_of_frames);
            counter++;
            faults++;
            frames[pos]=pages[i];
            time[pos]=counter;
        }

        for(j=0;j<no_of_frames;++j) printf("%d\t",frames[j]);
        printf("\n");
    }

    printf("\nTotal Page Faults = %d",faults);
    return 0;
}
```

OUTPUT

```
Enter number of frames: 3
Enter number of pages: 6
Enter reference string: 5 7 5 6 7 3
5 -1 -1
5 7 -1
5 7 -1
5 7 6
5 7 6
3 7 6

Total Page Faults = 4
```

RESULT

Thus, the LRU page replacement algorithm was implemented successfully and the total number of page faults was obtained.

EXPERIMENT 13

PAGE REPLACEMENT – OPTIMAL

Date: _____

AIM

To implement the Optimal page replacement algorithm.

ALGORITHM / PROCEDURE

1. Read the number of frames and the reference string.

2. Check whether the referenced page is already present.

3. If a free frame is available, place the page there.

4. Otherwise examine future references and select the page whose next use is farthest in the future, or which is not used again.

5. Replace the selected page and increment the page-fault count.

6. Display the final number of page faults.

PROGRAM

```
#include<stdio.h>

int frames[10], pages[50], nf, np;

int optvictim(int index)
{
    int i,j,far=-1,pos=0;

    for(i=0;i<nf;i++)
    {
        int found=0;
        for(j=index+1;j<np;j++)
            if(frames[i]==pages[j])
            {
                found=1;
                if(j>far) { far=j; pos=i; }
                break;
            }

        if(!found) return i;
    }
    return pos;
}

int main()
{
    int i,j,flag,pos=-1,faults=0;

    printf("Enter number of frames: ");
    scanf("%d",&nf);
    printf("Enter number of reference pages: ");
    scanf("%d",&np);
    printf("Enter reference string: ");

    for(i=0;i<np;i++) scanf("%d",&pages[i]);
    for(i=0;i<nf;i++) frames[i]=-1;

    for(i=0;i<np;i++)
    {
        flag=0;

        for(j=0;j<nf;j++)
            if(frames[j]==pages[i]) { flag=1; break; }

        if(!flag)
        {
            faults++;
            for(j=0;j<nf;j++)
```

```
                if(frames[j]==-1) { pos=j; break; }

            if(pos==-1) pos=optvictim(i);
            frames[pos]=pages[i];
        }

        for(j=0;j<nf;j++) printf("%d\t",frames[j]);
        printf("\n");
        pos=-1;
    }

    printf("Number of page faults: %d",faults);
    return 0;
}
```

OUTPUT

```
Reference string: 7 0 1 2 0 3 0 4
Frames: 3

The page frames are updated according to future references.
Number of page faults: depends on the entered reference string.
```

RESULT

Thus, the Optimal page replacement algorithm was implemented successfully.